System Architecture: AI-Powered Inventory Automation

Executive Summary

This document outlines the architecture of an AI-powered inventory management system designed to automate the traditionally manual process of handling supplier invoices and delivery notes. The system transforms physical documents, either scanned in-store or uploaded via WhatsApp, into structured digital data that is validated, corrected through a conversational AI, and integrated with the store's back-office systems. The architecture is designed to be scalable, reliable, and cost-efficient, with a strong emphasis on a seamless user experience for non-technical users.

Core Architectural Principles

The system is built on a foundation of key principles that guide its design and evolution:

1. **Scalability & Reliability:** The system is built on an asynchronous, queue-based architecture. This allows it to handle a high volume of concurrent operations reliably, with persistent jobs ensuring graceful recovery from failures.
2. **Human-in-the-Loop AI:** The system automates repetitive tasks but leverages a conversational interface (WhatsApp) to bring in a human user for critical validation and exception handling, ensuring high accuracy and building user trust.
3. **Efficient Context Management:** The system isolates media analysis from the main conversation. Specialized models process large files like images and audio, and only their text-based results are added to the AI's context. This ensures a lean, focused, and cost-effective conversational history.
4. **Self-Improving System:** The system is designed to learn from user interactions. Manual corrections and product matches from one invoice are recorded and used as a high-priority data source for automating the processing of future, similar documents.

High-Level System Architecture

The system is organized into several logical layers, representing the journey of a document from physical capture to digital integration.

1. **Ingestion Layer:** This is the entry point for all documents. It supports multiple channels, including:
    - **Hardware-based Scanning:** On-site scanners connected to IoT devices (Raspberry Pi) capture physical documents and securely upload them to the backend.
    - **Direct Digital Upload:** Users can send PDF documents directly via the system's WhatsApp business number. A unified "onboarding" service ensures all documents, regardless of their source, are processed and stored consistently.
2. **Storage Layer:** A multi-tiered storage strategy is used to ensure durability, fast access, and AI-readiness.
    - **Durable Object Storage (AWS S3):** The original PDF file is stored for long-term archival and auditing.
    - **Metadata Database (MongoDB):** All metadata about the document, its processing state, and related conversation history is stored in a NoSQL database for fast and flexible querying.
    - **AI Service Storage (OpenAI Files API):** The document is also uploaded to the AI provider's internal storage, making it efficiently accessible to vision models without needing to be resent with every request.
3. **Processing Core (Node.js, Bull.js, Redis):** This is the heart of the system.
    - It uses a **job queueing system (Bull.js with a Redis backend)** to manage a pipeline of asynchronous tasks for each document (e.g., initial validation, OCR extraction, data matching, inventory update).
    - The pipeline is dynamic and configurable on a per-client basis, allowing for different processing workflows.
4. **AI Orchestration Layer:** This layer is responsible for integrating with AI models (e.g., OpenAI) and managing the intelligent aspects of the workflow.
    - It uses a **multi-model strategy** (text, vision, audio) to optimize for cost and performance.
    - It employs a **tool-calling (function-calling) paradigm**, where a primary text model acts as an orchestrator. To ensure relevance and prevent erroneous actions, the specific tools made available to the AI are dynamically selected based on the document's current stage in the processing pipeline.
5. **Communication Gateway (WhatsApp):** This layer manages all user interactions.
    - It provides a conversational UI that requires zero training for the end-user.
    - It supports flexible user input, allowing users to interact via either text or voice messages, which are automatically transcribed for processing.

- It includes sophisticated logic for managing conversation state and ensuring a seamless, linear user experience, even when multiple documents are being processed in the background.

Key Design Patterns and Solutions

Challenge: Handling Conversational Concurrency

A critical challenge is managing the user experience when a user uploads multiple documents in quick succession. While the backend should process these in parallel for efficiency, the user-facing conversation must focus on one document at a time to avoid confusion.

- **Solution: The Conversation Manager** This central service acts as an orchestrator for the user's conversational context. It maintains a strict FIFO queue of "active" documents for each user. Each document has its own dedicated outbound message queue, which is paused by default. The Conversation Manager ensures that only the message queue for the currently active document is running. Messages for all other documents are buffered in their respective paused queues. This pattern effectively decouples the high-performance, concurrent backend processing from the simple, serialized, and predictable conversational interface presented to the user.

Isolated and Dynamic AI Context

To ensure accuracy and prevent information from one document's processing from "bleeding" into another's, the system does not maintain a single, long-running AI memory. Instead, each background processing job, before interacting with the AI, dynamically queries the database to construct a clean conversational history containing only the messages relevant to the specific document it is currently processing. This "just-in-time" context assembly guarantees that every AI task operates in a completely isolated and focused environment.

Challenge: Ensuring Robust and Orderly Communication

- **Solution: Per-User Queues & Message Debouncing** A debouncer service handles message bursts, while dedicated, single-concurrency queues for each user guarantee strict chronological processing to maintain conversational integrity.

Challenge: Handling Noisy and Unstructured OCR Data

Raw data from Optical Character Recognition (OCR) services can be unreliable, sometimes including "hallucinated" tables from document headers or footers, or misinterpreting multi-page

table structures.

- **Solution: A Two-Stage OCR & Validation Process** The system employs a two-stage approach to ensure data quality.
  1. **Extraction:** A high-resolution OCR service first converts the document image into structured JSON data.
  2. **AI-Powered Sanitation:** This raw output is then passed to a specialized AI model that identifies and discards “junk” tables, ensures header consistency across multi-page documents, and flags any structural anomalies it cannot resolve with high confidence. This step determines whether the document can proceed automatically or if it requires human review.

Challenge: Accurate Product Matching in a Morphologically Rich Language

Matching OCR’d item names from an invoice to a product catalog is a non-trivial task, especially in Hebrew, where prefixes and word forms change the text.

- **Solution: A Multi-Pass Matching Cascade** A sequence of matching strategies is employed, ordered from most to least certain. An item resolved by one pass is excluded from subsequent ones.
  1. **History Pass:** The system leverages a Lucene-enhanced index in **MongoDB Atlas Search** to perform a powerful query against a historical database of previously approved manual matches. This allows it to learn directly from the user’s past decisions.
  2. **Barcode Pass:** An exact match on the item’s barcode provides the most reliable identification.
  3. **Semantic Search Pass:** For remaining items, a vector embedding is generated from the item name, and a similarity search is performed against a pre-embedded product catalog to find semantically similar products.
  4. **Lemmatization Pass:** This pass performs a powerful keyword search to complement the semantic search. It operates on an index of pre-computed linguistic roots (lemmas) within **MongoDB Atlas Search**, using relevance-scoring algorithms like **BM25** to accurately match different morphological word forms (e.g., matching the construct-state “גבינת” to its root “גבינה”). This search mechanism is also reused to power product lookups during the user correction loop, after inventory update drafts are generated.
  5. **AI Resolution Pass:** Finally, an AI model reviews the original item name against the list of candidates generated by the semantic and lemmatization passes to select the most likely match.
  6. **User Review and Draft Generation:** For the final user review, a draft PDF is dynamically generated on the backend using React (SSR) and a headless browser to create high-quality, component-based documents.

Challenge: Integrating with Legacy Back-Office Systems

Many target businesses use older back-office systems that lack modern, well-documented APIs for tasks like authentication or data synchronization.

- **Solution: Headless Browser Automation and Intelligent Caching**
  - For authentication, a **headless browser (Puppeteer)** is used to programmatically perform a user login, simulating a real user to extract a session token.
  - For data like the product catalog, the system performs an **intelligent delta-based synchronization**. It periodically fetches the full catalog from the legacy system's API, compares it against its local cache using an MD5 fingerprint of each product, and then only performs the necessary database operations (inserts, updates, deletes). This, combined with on-demand embedding generation only for new/updated items, minimizes both API load and AI processing costs.

Technology Stack

Category	Technology	Justification
Backend	Node.js, TypeScript	Provides a highly performant, non-blocking I/O model suitable for an asynchronous, I/O-bound application. TypeScript adds crucial type safety.
Job Queuing	Bull.js, Redis	A mature and reliable system for managing persistent, asynchronous background jobs, essential for the system's core processing pipeline.
Database	MongoDB Atlas	Its flexible, document-based model is ideal for storing varied metadata, and its integrated Atlas Search feature powers advanced text-search capabilities.
Object Storage	AWS S3	A durable, scalable, and cost-effective solution for long-term storage of original source documents.

Category	Technology	Justification
AI & NLP	OpenAI models, Custom Microservice	Leverages state-of-the-art models for orchestration, vision and embeddings, supplemented by a self-hosted microservice for specialized linguistic tasks.
Deployment	Docker, Docker Compose, GitHub Actions	A containerized approach ensures consistency across development and production environments, with CI/CD for automated builds and deployments.
Hardware/IoT	Raspberry Pi, Python	A cost-effective and flexible platform for managing on-site hardware and providing a bridge to the cloud backend.